

ОТЧЕТ
о выполнении практической работы №1
Пользовательские запросы в веб-приложении (DevTools,
HTTP, REST, JSON, Postman)

Дисциплина «Обеспечение безопасности веб-приложений»

Тема: «Оптимизация веб приложений»

Специальность: 09.02.07 Информационные системы и
программирование

Квалификация: Разработчик веб и мультимедийных приложений

Выполнил: Бойко М.А

Студент группы: 4ИСИП-923

Проверил:

Преподаватель: Мясников Н.Р.

Дата: «___» _____ 2026 г.

Оценка: _____

2. Задание

<https://jsonplaceholder.typicode.com/posts>

Скриншот списка запросов.

Name	Status	Type	Initiator	Size	Time
 posts	304	docume...	Other	0.1 kB	199 ms
 favicon.ico	200	x-icon	Other	(disk ca...	1 ms

2.3. Детальный разбор одного запроса

Выберите любой запрос и опишите:

метод;

URL и параметры;

заголовки (Request Headers / Response Headers);

тело ответа (Response / Preview);

код ответа;

время выполнения.

метод	GET
URL	https://jsonplaceholder.typicode.com/posts
Код ответа	304
Размер данных	0.1KB
Время загрузки	199ms

Сравните его с другим запросом и сделайте вывод.

URL запроса

jsonplaceholder.typicode.com/posts/3

метод	GET
URL	https://jsonplaceholder.typicode.com/posts/3
Код ответа	200
Размер данных	0.7KB
Время загрузки	196ms

2.4. Эксперименты с запросами

Выполните в консоли:

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(res => res.json())
  .then(data => console.log(data));
```

Проанализируйте запрос во вкладке **Network**. Сравните:

GET /posts (все посты);

GET /posts/1 (один пост).

Сделайте выводы по:

размеру ответа;
времени загрузки;
количеству записей.

Posts1

Размер ответа	76B
Время загрузки	76ms
Количество записей	1

Posts

Размер ответа	754B
Время загрузки	194ms
Количество записей	100

2.5. Работа с параметрами

1. Откройте в браузере:

<https://jsonplaceholder.typicode.com/posts?userId=1>

1. Сравните с <https://jsonplaceholder.typicode.com/posts>:

	Posts?	posts
Кол-во объектов	10	100
Размер ответа	1.4kB	76B
Время загрузки	67ms	72ms

2. сколько объектов вернулось;

3. размер ответа;

4. время загрузки.

Сделайте вывод, как параметры помогают **уменьшить объём данных**.

Ответ: Чем больше фильтров тем меньше объем данных.

2.6. Массовые запросы

Откройте несколько URL одновременно:

- `/posts/1`
- `/posts/2`
- `/posts/3`

Зафиксируйте:

как меняется время отклика;

как браузер параллелит запросы;

нет ли избыточных запросов.

	<code>/posts/1</code>	<code>/posts/2</code>	<code>/posts/3</code>
Время отклика	1.47ms	1.47ms	1.48ms

Ответ: Браузер параллелит запросы подгружая языки параллельно для трех posts

Ответ: Избыточных запросов нет.

2.7. Заголовки и ответы

1. Во вкладке **Headers** выпишите не менее 5 ключевых заголовков

Ответ:

`Content-Type` - формат передаваемых файлов (`application/json; charset=utf-8`)

`access-control-allow-credentials` – заголовок ответа сервера(`true`)

`ETag` – идентификатор версии ресурса(`W/"124-yiKdLzqO5gfBrJFrcdJ8Yq0LGnU"`)

`Date` – дата запроса(`Wed, 09 Sep 2026 11:38:15 GMT`)

`Server` – название сервера(`cloudflare`)

2. Сравните вкладки **Preview** и **Response**:

3. чем они отличаются по представлению данных;

Ответ:

Preview показывает интерпретированные данные

Response показывает исходные данные

4. в каких случаях удобнее использовать Preview, а в каких — Response

Ответ:

Response удобнее когда нужно увидеть точный текст ответа

Preview удобнее когда надо быстро посмотреть структуру JSON

2.8. Оптимизация запросов

Сравните ответы:

Запрос	Размер ответа	Время загрузки
<code>/posts</code>	...	...
<code>/posts?userId=1</code>	...	...
<code>/posts/1</code>	...	...

Предложите меры оптимизации:

пагинация (ограничение количества записей на страницу);

фильтрация (по `userId`, статусам и т.п.);

уменьшение размера ответа (убрать лишние поля);

кэширование на клиенте и сервере;

использование правильных заголовков (`Cache-Control`, `ETag` и т.д.).

2.9. Работа с API через Postman

1. Перейдите на сайт:

<https://www.postman.com/>

1. Выполните:

2. `GET /posts`

3. `GET /posts/1`

Сравните ответы.

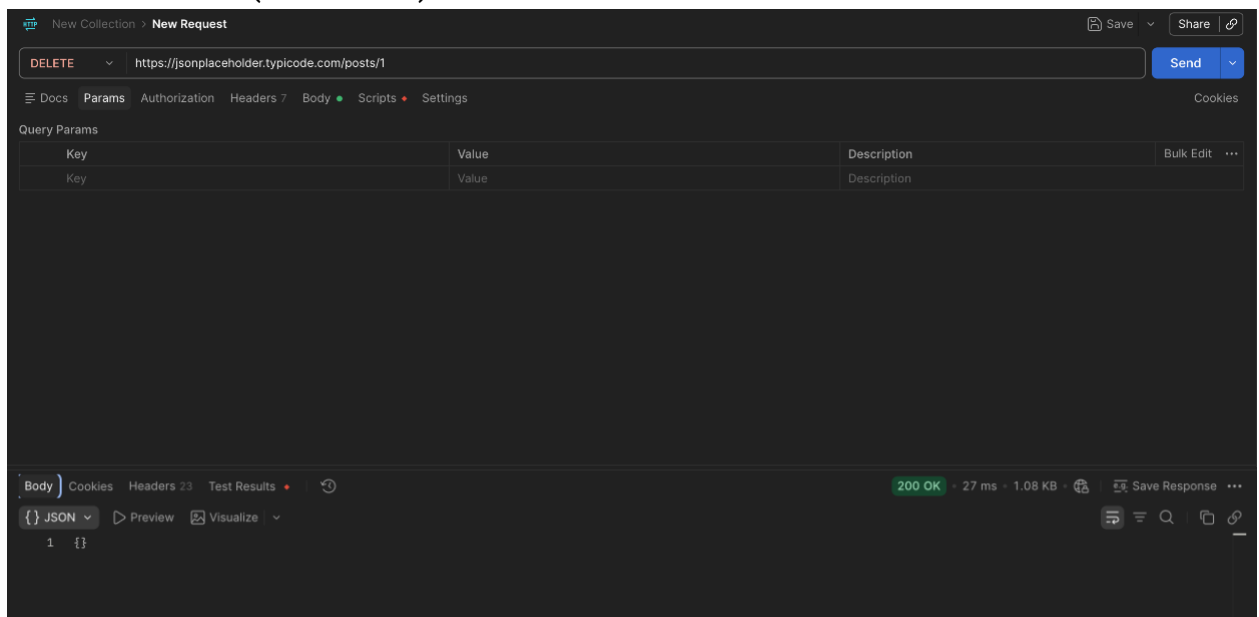
	posts	Posts1
Время отклика	244ms	218
Размер ответа	28kb	1.41kb

1. Выполните **POST** `/posts` с JSON-телом:

```
{
  "title": "Пост через Postman",
  "body": "Текст поста",
  "userId": 2
}
```

1. Выполните:
2. **PUT** `/posts/1` — полное обновление ресурса;
3. **PATCH** `/posts/1` — частичное обновление (изменить только один-два поля).
Объясните, чем отличаются результаты.

1. Выполните **DELETE** `/posts/1` и зафиксируйте:
2. код ответа;
3. тело ответа (если есть).



2.10. Сравнение DevTools и Postman

Сделайте таблицу или краткое сравнительное описание:

DevTools

анализ реальных запросов страницы;
проверка производительности, кэширования;
работа в контексте открытого сайта.

Postman

ручное конструирование запросов;
сохранение коллекций;
тесты и скрипты;
работа с авторизацией и токенами.

Критерий	DevTools	Postman
анализ реальных запросов страницы	Показывает все запросы, которые отправляет сайт	Не видит запросов сайта
проверка производительности, кэширования	Вкладки Performance и Network	Не предназначен для этого
работа в контексте открытого сайта	Использует куки и сессию	Свой контекст
ручное конструирование запросов	Нельзя собрать запрос с нуля	Полный редактор
сохранение коллекций	Нет, данные живут только когда открыта вкладка	Есть коллекции, папки
тесты и скрипты	Можно писать на JS в консоли	Скрипты и тесты на JS
работа с авторизацией и токенами	Только просмотр того, что отправлено	Bearer, Basic, API Key
Назначение	Наблюдать и анализировать	Тестировать, сохранять
Где работает	Внутри браузера	Отдельное приложение

Сделайте вывод: **когда удобнее DevTools, а когда Postman.**

Ответ:

DevTools удобнее когда нужно посмотреть производительность, кэш, ошибки на живо страницы

Postman удобнее когда нужно создать запрос вручную с нуля, писать тесты и скрипты

3. Контрольные вопросы

1. Что такое DevTools и какие задачи решают вкладки Elements, Console и Network?

Ответ: DevTools - встроенный в браузер инструмент разработчика. Elements показывает HTML/CSS. Console выводит ошибки и дает возможность выполнять JS команды. Network показывает все сетевые запросы (URL, заголовки, тело и тд)

2. Как открыть DevTools и какие настройки вкладки Network важны для замеров производительности?

Ответ: нажать f12, включить фаст 4g, включить preserve log

3. Перечислите основные части HTTP-запроса (метод, URL/параметры, заголовки, тело) и их назначение.

Ответ: метод(что делаем), URL и параметры(куда и с какими данными обращаемся), заголовки(метаданные)

4. Чем отличаются методы GET, POST, PUT, PATCH и DELETE? Приведите по одному практическому примеру.

Ответ: GET(получить данные с сервера(GET /posts), POST создать(POST /posts), PUT заменить целиком(PUT /posts/1), PATCH изменить частично (PATCH /posts/1), DELETE удалить(DELETE /posts/1)

5. Для чего используются методы HEAD и OPTIONS и где встречается preflight-запрос?

Ответ: HEAD похож на GET, но только проверяет заголовки, OPTIONS - узнать доступные методы, preflight - автоматический OPTIONS запрос

6. Расскажите о группах кодов ответов (1xx–5xx) и приведите примеры для 200, 301/302, 304, 404, 500.

Ответ :

100 - информационные

200 - успех

300 - перенаправление

400 - ошибка на стороне клиента

500 - ошибка сервера

7. Что такое REST и как ресурс в REST представлен на уровне URL?

Ответ: REST - архитектурный стиль для API на основе HTTP

8. Какие типы данных поддерживает JSON? Приведите короткий пример JSON-объекта и массива.

Ответ: строка, число, логическое, null, объект, массив

Пример {"name": "Иван", "age": 25}

[1, 2, 3]

9. В чём отличие вкладок Preview и Response в карточке запроса DevTools?

Ответ: Preview - отформатированный просмотр

Response - сырой ответ

10. Какие заголовки чаще всего анализируются при оптимизации (например, Content-Type, Cache-Control, ETag, Authorization)?

Ответ: Content-Type - тип данных, Cache-Control - правила кэширования, ETag - версия ресурса, Authorization - токен доступа

11. Какие приёмы снижения нагрузки на сеть можно использовать для API: пагинация, фильтрация, кэширование и др.?

Ответ: пагинация, фильтрация и сортировка на сервере, кэширование, сжатие

12. Чем работа с API в Postman отличается от анализа запросов в DevTools, и когда уместен каждый инструмент?

Ответ: DevTools смотреть производительность, кэш, работать в контексте страницы, Postman делать запросы вручную, писать тесты, хранить коллекции. каждый уместен в зависимости от задачи. Для анализа DevTools, для разработок и тестов Postman

Чек-лист для самопроверки

Баллы	Критерии выполнения
-------	---------------------

7	Выполнены все пункты практики: – первичный анализ запросов (таблица + скриншоты); – детальный разбор + сравнение запросов; – работа с параметрами, массовые запросы, заголовки (5+); – таблица оптимизации (/posts, /posts?userId=1, /posts/1); – эксперименты в Postman (GET, POST, PUT, PATCH, DELETE); – сравнение DevTools и Postman; – все обязательные скриншоты (Network, Console, Postman); – есть итоговые выводы и рекомендации; – аккуратное оформление: таблицы, структурированные заголовки, выводы.
6	Выполнена полная практическая часть в DevTools (таблицы, скриншоты, сравнения); реализованы все виды запросов в Postman: GET, POST, PUT, PATCH, DELETE; есть сравнение DevTools и Postman; составлены рекомендации по оптимизации (5–7 мер); оформление есть, но структура слабее: не всегда удачно выбраны заголовки, таблицы оформлены неаккуратно.
5	Выполнены основные шаги анализа запросов: таблица с методами, URL, кодами ответа и временем; есть сравнение /posts и /posts/1, работа с параметрами; сделано 2–3 скриншота; разобраны заголовки и показаны примеры запросов с параметрами; примеры оптимизации приведены, но не оформлены отдельной таблицей; в Postman реализованы

	только GET/POST .
3–4	Краткое описание DevTools и HTTP-запросов; выполнен первичный анализ запросов (таблица: метод, URL, код ответа, размер, время); сделан хотя бы один скриншот из DevTools; есть разбор одного запроса, но без сравнений и выводов; JSON/REST API упомянуты, но Postman не использован или использован формально.
1–2	Даны лишь отдельные ответы без общей структуры; скриншоты отсутствуют или не соответствуют заданию; теоретическая часть слабо раскрыта, практическая часть выполнена формально.
0	Работа выполнена с помощью ИИ (автоматически сгенерированные ответы без личных скриншотов и анализа); отсутствует выполнение практических шагов, присутствует только копипаст из методических материалов.